

LogicConnect: Structured Connectivity Design for Differentiable Logic Gate Networks

Abstract—Differentiable logic gate networks learn Boolean circuits whose connectivity determines which signals can be combined during gate-function optimization. Learning these connections can improve accuracy, but increases the memory required to process candidate inputs during training. In this paper, we present LogicConnect, a structured connectivity method that constructs the graph before training while preserving the network’s gate budget. LogicConnect combines semantic input pairing with regular multiscale matchings in deeper layers and supports dense and convolutional architectures without trainable routing parameters. We evaluate the method on MNIST, Fashion-MNIST, and CIFAR-10 through accuracy comparisons, training-resource measurements, and structural ablations. LogicConnect improves dense CIFAR-10 test accuracy by 3.18–4.59 percentage points across three model sizes and three paired seeds. Convolutional S/M models improve by 3.26/2.08 points in single-seed tests, although the three-seed S validation comparison remains inconclusive. Compared with learned Top-32 routing, LogicConnect uses 93.7–94.2% less peak GPU memory and 80% fewer trainable parameters, with an accuracy reduction of 1.84–5.30 points. Degree-preserving ablations identify first-layer pairing as the strongest measured dense contribution, supporting structured connectivity as a design choice for improving accuracy under constrained training memory.

I. INTRODUCTION

Differentiable Logic Gate Networks (DLGNs) provide an alternative path toward efficient machine learning by learning directly with the primitive operations used in digital hardware. Rather than relying on multiply-accumulate operations, a DLGN is composed of Boolean gates such as AND, OR, and XOR. During training, these discrete operations are represented through differentiable relaxations, allowing the network to be optimized with gradient descent; after training, each relaxed gate is hardened to a Boolean function for efficient inference [1]. This direct correspondence between the learned model and Boolean computation makes DLGNs particularly attractive for resource-constrained and hardware-oriented machine learning.

A fundamental characteristic of this architecture, however, is its sparse connectivity. Each conventional DLGN gate has only two inputs. In the original formulation, these two predecessors are selected before training and remain fixed, while optimization determines which Boolean function the gate should implement [1]. Therefore, training can change *how* two signals are combined, but not *which* signals meet at a gate. If two informative signals are never connected through the available

graph, learning the gate functions alone cannot create that missing edge. At a fixed depth and width, connectivity consequently determines the combinations of information that can be formed throughout the network and becomes part of the effective architectural capacity of the DLGN.

Recent work increasingly supports this view of connectivity as an independent design dimension. BitLogic, for example, separates connectivity from the gate parameterization, fan-in, input encoding, and output head when studying the design space of LUT- and logic-native neural networks [2]. Other approaches go further and optimize the predecessors themselves during training [3], [4]. Learned connectivity can recover connections that were absent from the original graph and can therefore improve accuracy or reduce the number of required gates. This flexibility, however, is not free. During training, each gate must consider multiple candidate signals, requiring additional routing state and candidate activations even though the final hardened network may retain only a small number of predecessors. This creates a practical design choice between inexpensive but arbitrary *fixed random connectivity* and more flexible but substantially heavier *learned connectivity*.

The connectivity question becomes even more important for vision workloads. Early DLGNs operate on flattened image representations, where random connections do not explicitly preserve the strong spatial and channel relationships present in images [1]. Convolutional DLGNs address this limitation by introducing shared logic-gate trees that operate over local receptive fields, enabling logic networks to capture spatial patterns and scale to substantially more challenging vision tasks [5]. Yet convolution does not remove the connectivity design problem; instead, it changes its form. A convolutional logic tree must select both *where* to sample within its receptive field and *which input channels* provide those samples. The spatial pattern is reused across image locations through weight sharing, making arbitrary changes to this structure undesirable. Thus, a useful connectivity method should respect the convolutional spatial sampler while improving how channels are paired. A shared connectivity principle must therefore accommodate two distinct representations: individual input coordinates in dense DLGNs and shared channel groups in convolutional DLGNs.

Before designing such a method, we ask whether a simpler property—balanced predecessor usage—is already sufficient. As a motivational experiment, we consider a dense CIFAR-10 network whose deeper connectivity is kept fixed and structured. We then randomize only the first-layer predecessor identities while preserving, for every input slot, exactly how often each predecessor is used. The resulting graph therefore maintains the

same predecessor degrees, but forms different input pairs. This seemingly small change reduces hardened validation accuracy by 4.49 percentage points, with a paired 95% confidence interval of [3.90, 5.09] over three seeds. Since the native dense random construction already provides broadly balanced fan-out, this experiment shows that *which signals are paired matters beyond how frequently each signal is used*. It motivates exploiting signal identity when constructing the graph rather than treating predecessor indices as interchangeable.

Based on this observation, we propose **LogicConnect**, an offline connectivity construction for dense and convolutional DLGNs. LogicConnect first introduces structure where signal identity is most explicit. For dense image inputs, it forms semantic first-layer pairs according to relationships among the encoded image signals rather than arbitrary input indices. In subsequent layers, LogicConnect organizes predecessor interactions using regular multiscale matchings. Complete matching stages are selected primarily to balance predecessor usage, while an ancestry-overlap criterion distinguishes alternatives with equivalent degree balance. Once constructed, the graph is frozen and conventional DLGN training learns only the gate functions.

The same principle is applied to convolutional DLGNs without replacing their convolutional execution model. LogicConnect modifies the channel pairs available to the shared logic trees while retaining the existing spatial sampler and parameter sharing of the convolutional architecture [5]. The classifier is then constructed using the same fixed multiscale connectivity principle employed by the dense network. Thus, LogicConnect neither introduces a new inference operator nor increases the declared depth, width, or gate budget. Instead, it moves connectivity design offline, seeking a more useful fixed graph without creating an additional trainable routing problem.

This leads to the central question investigated in this work: *how much accuracy can a DLGN recover through structured offline connectivity, under the same declared architecture, before the additional training cost of learned routing becomes necessary?* Our main contributions are:

- We propose **LogicConnect**, a shared fixed-connectivity construction for dense and convolutional DLGNs. It combines semantic input pairing with regular multiscale predecessor matchings, while preserving the conventional fixed-index representation and introducing no trainable routing parameters.
- We quantify the resulting **accuracy–training-resource trade-off**. Across three matched dense CIFAR-10 network sizes, LogicConnect improves hardened test accuracy over fixed random connectivity by 3.18–4.59 percentage points across three paired seeds. Compared with the evaluated Top-32 learned-routing configuration, it uses 93.7–94.2% less peak GPU memory while remaining within 1.84–5.30 accuracy points. We further evaluate the same connectivity principle in convolutional DLGNs; the held-out single-seed results are positive, while replicated validation results remain inconclusive.
- We perform **structural attribution** to identify where the

gains originate. Degree-preserving experiments separate predecessor identity from predecessor frequency and show the strongest measured dense effect at the first layer. Additional experiments examine the incremental effect of deeper connectivity, simpler construction rules, alternative gate training, and distribution shift, characterizing both the benefits and the limits of structured fixed connectivity.

II. RELATED WORK

Research on DLGNs has addressed the design of the network architecture, the selection of gate inputs, and the optimization of Boolean functions. Deep DLGNs introduced gradient-based gate learning with fixed random connections, followed by discretization for Boolean inference [1]. Convolutional DLGNs extended this approach with shared logic trees, pooling, and residual initialization to exploit image structure [5]. Studies of output aggregation further show that temperature and classifier design affect scaling to larger class counts [6]. These architectural choices determine how signals are represented and combined; LogicConnect designs the connections within a given dense or convolutional architecture.

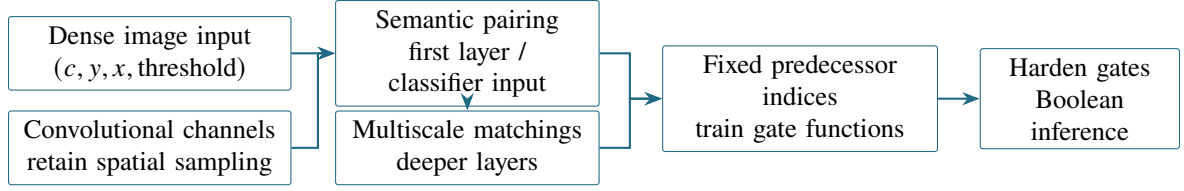
Learning the connections provides another means of improving the combinations available to each gate. Mommen et al. optimize distributions over candidate predecessors, allowing input selection to change during training [3]. LILogic Net combines sparse Top-K routing with basis projection to improve compact logic networks [4]. The resulting accuracy gains are accompanied by additional training state for candidate selection. LogicConnect addresses this trade-off through an offline construction whose connections remain fixed during gate learning. We therefore compare the methods in terms of both classification accuracy and the memory, parameter count, and time required for training.

A complementary line of work improves the optimization of the gate functions themselves. Mind the Gap uses Gumbel noise and a straight-through estimator to reduce the loss caused by discretization [7]. Light DLGNs modify the gate parameterization, WARP represents Boolean functions in a Walsh–Hadamard basis, and multilinear polynomial fitting examines redundant optimization directions in soft gate mixtures [8]–[10]. These methods act on the functions computed from the assigned inputs. Our main experiments hold the gate-training rule fixed to isolate connectivity, while additional WARP experiments examine its compatibility with an alternative parameterization.

Connectivity also interacts with broader choices in logic-network design. BitLogic studies encoding, connectivity, fan-in, and training within a framework for lookup-table-native networks [2]. Unit tying reduces trained circuit size through functional approximation and refinement [11]. These approaches concern a broader design space or a later optimization stage than the fixed-budget construction studied here.

III. LOGICCONNECT METHODOLOGY

LogicConnect uses the input representation and layer dimensions to construct connections before training, as shown in Figure 1. Semantic pairing exploits known relationships between encoded image values, while multiscale matching extends the construction to hidden signals whose spatial meaning is no longer



Stage priority: degree spread $\rightarrow$ ancestry novelty $\rightarrow$ nominal scale

Fig. 1: Overview of LogicConnect. Semantic pairing constructs input connections, and multiscale matching organizes deeper connections. The convolutional construction preserves spatial sampling and shared logic trees. Training optimizes gate functions on the resulting fixed graph.

explicit. Both components produce fixed predecessor indices, allowing training with the original gate parameterization.

A. Network representation

For a layer l containing m_l two-input gates and n_l available predecessors, we represent connectivity by $I_l \in \{0, \dots, n_l - 1\}^{2 \times m_l}$. Gate j receives distinct predecessors $a = I_l[0, j]$ and $b = I_l[1, j]$. Following the baseline DLGN formulation [1], its relaxed output is

$$h_{l,j} = \sum_{q=0}^{15} \text{softmax}(\theta_{l,j})_q \tilde{f}_q(h_{l-1,a}, h_{l-1,b}). \quad (1)$$

Here, $\theta_{l,j}$ contains the trainable gate logits and $\tilde{f}_q$ denotes the continuous relaxation of Boolean function q . Training optimizes θ on the fixed graph, and discretization selects the highest-weight function for each gate. LogicConnect determines I_l while preserving this optimization, the loss function, and output aggregation.

We associate each predecessor i with a structural ancestry set A_i describing its accessible sources. At a dense image input, labels identify raw channel-pixel values, with thermometer thresholds of the same value sharing a label. Propagating $A_j = A_a \cup A_b$ records sources that can reach each gate through its connections. The learned gate function determines which accessible inputs actually affect the output.

B. Semantic input pairing

The image encoding provides useful information that is lost when inputs are treated only as numbered bits. In particular, thresholds of the same pixel-channel value share a source, whereas bits from different positions or channels can convey distinct image information. We use this distinction to construct candidate pairs from channel, spatial, and threshold metadata, excluding pairs whose two bits come from the same raw source.

Spatial candidates connect horizontal and vertical positions at power-of-two strides up to half the corresponding axis length, using cyclic boundaries. Additional candidates at the finest scale connect aligned channels and combine different thresholds with a spatial displacement. The construction cycles through these candidate stages, distributing connections across the available relationships while controlling how often each predecessor is selected.

Within each stage, candidates are ordered by the sum of endpoint fan-outs, then their maximum fan-out, and finally their indices. The initial selection examines up to 64 unused candidates in a seeded affine order; once distinct pairs are exhausted, repeated pairs follow the same degree priority. This bounded search preserves a deterministic construction while limiting the cost of selecting semantically defined pairs.

C. Multiscale matching in hidden layers

Deeper signals lack a direct image-coordinate interpretation, so we construct their connections through regular matchings over predecessor indices and use ancestry to distinguish their accessible sources. Each matching contains disjoint pairs, allowing a predecessor to participate at most once per stage. Varying the matching changes which signals are combined while controlling predecessor reuse.

For n predecessors, the construction considers $K = \lceil \log_2 n \rceil$ scales. When n is a power of two, scale s pairs index i with $i \oplus 2^s$, providing connections at different index separations. For other widths, an affine traversal with a coprime step pairs adjacent elements; at odd widths, the unmatched predecessor rotates. Each complete stage contains $\lfloor n/2 \rfloor$ pairs. If fewer gates remain, a partial stage selects disjoint pairs by endpoint degree sum, maximum degree, and index.

We select stages by prioritizing balanced reuse and then considering the overlap between their ancestry sets. Let d_i be the current fan-out of predecessor i , and let $\delta_i(P_s)$ count its occurrences in the pairs retained from candidate stage s . The resulting degree spread is

$$D_s = \max_i (d_i + \delta_i(P_s)) - \min_i (d_i + \delta_i(P_s)). \quad (2)$$

Minimizing D_s gives preference to stages that maintain uniform access to preceding signals. Among candidates with the same spread, we favor pairs with less relative ancestry overlap, measured by

$$N_s = \frac{1}{|Q_s|} \sum_{(a,b) \in Q_s} \left(1 - \frac{|A_a \cap A_b|}{\max(1, \min(|A_a|, |A_b|))} \right). \quad (3)$$

The set Q_s contains the complete candidate matching, or 2,048 evenly indexed pairs when the matching is larger. We use the same set when selecting a partial stage. Identical nonempty ancestry sets contribute zero to this score, while disjoint sets

Algorithm 1 Deeper-layer stage selection

Require: Ancestry A , width m , layer l , seed
Initialize degrees $d = 0$, output pairs $I = \emptyset$.
while $|I| < m$ **do**
 Generate the K seeded scale matchings M_s .
 Form degree-ordered partial stages P_s if needed.
 Compute D_s and N_s using (2)–(3).
 Among unused scales, minimize D_s , then maximize N_s ;
 break ties by nominal scale order.
 Append P_s ; update d and mark scale s used.
 Reset used scales after a complete K -stage cycle.
end while
Propagate ancestry by union; return fixed indices I .

contribute one. LogicConnect minimizes D_s and then maximizes N_s , resolving remaining ties by cyclic nominal scale order. Every scale is used once per cycle, as summarized in Algorithm 1. Ancestry overlap is a structural heuristic whose contribution to accuracy is evaluated separately.

For even widths, complete matchings increment every degree equally, and a final partial matching increments each degree at most once. Starting from uniform degrees therefore gives a final spread of at most one. For example, eight predecessors feeding ten gates use two complete matchings and two additional pairs, producing degrees of two or three. Degree-preserving ablations test the accuracy contribution of pair identities beyond this balance property.

D. Extension to convolutional networks

Convolutional DLGNs share logic trees across spatial positions, requiring the construction to preserve both channel organization and spatial sampling [5]. We apply LogicConnect to channel pairing while retaining the original receptive-field sampler, depth-three logic trees, weight sharing, and OR pooling. The first channel assignment uses semantic metadata, and subsequent channel groups use multiscale matching. This formulation changes connectivity at the channel level without replacing the convolutional computation with independent dense gates.

Channel ancestry initially uses encoded-channel identities. We add an identity label to each output channel so that channels remain distinguishable when their upstream support sets become identical. After flattening, classifier inputs retain this ancestry and receive spatial-feature identities; semantic pairing constructs the classifier input connections, followed by deeper matching stages. These labels support a common construction at different signal granularities, rather than representing exact pixel-dependence sets.

E. Construction overhead

Construction uses packed ancestry bitsets; CPU storage grows with layer width and label universe. Each stage decision considers K matchings and evaluates at most 2,048 ancestry pairs per candidate. We measure this one-time cost separately from training. A dense layer retains $16m_l$ trainable gate logits and $2m_l$ fixed indices, discarding ancestry before deployment. The

circuit retains its logical budget; physical area and interconnect cost depend on synthesis and placement.

IV. EXPERIMENTAL EVALUATION

A. Experimental setup

We evaluate LogicConnect on MNIST [12], Fashion-MNIST [13], and CIFAR-10 [14], using the configurations in Table I. The evaluation covers accuracy, training requirements, structural ablations, and transfer. Experiments use RTX PRO 6000 Blackwell Max-Q GPUs with PyTorch 2.9.0 and CUDA 13.0.

For each connectivity comparison, we match the architecture, encoding, gate parameterization, optimizer, training updates, evaluation cadence, initialization, and training seed. The dense random baseline balances predecessor fan-out, whereas the convolutional baseline cycles through adjacent channel groups and samples spatial connections randomly. We reserve 10% of the training data for validation using split seed 2027, select checkpoints by hardened validation accuracy, and freeze them before test evaluation. Hardened accuracy is measured after selecting a discrete Boolean function for each gate.

Unless otherwise stated, experiments use paired seeds 0, 1, and 2. We report means and sample standard deviations, and compute 95% confidence intervals for accuracy differences using paired Student- t statistics. Resource measurements comprise peak PyTorch GPU allocation, trainable parameters, CPU construction time, and training wall time including evaluation. Tables distinguish our methods (OUR), reproduced baselines (REP), and published values (R), with V and T identifying validation and test results.

LogicConnect (LC) refers to the unified construction in Section III. LogicConnect-D (LC-D) is a dense specialization based on semantic butterfly pairing and ancestry-based swaps. Its results are reported separately, and boldface identifies the largest observed local mean.

B. Classification accuracy

Table II compares the dense networks at identical gate budgets. LogicConnect improves CIFAR-10 test accuracy by 3.18, 4.56, and 4.59 percentage points for S, M, and L, respectively. Each improvement is positive for all three paired seeds, and all three confidence intervals exclude zero. The gain therefore persists across the evaluated model sizes without increasing the number of gates or changing gate optimization. MNIST also improves by 0.66 points with a positive confidence interval; the 0.52-point Fashion-MNIST improvement remains inconclusive because its interval includes zero.

The unified construction remains competitive with LC-D, although neither has the highest mean in every configuration. LC-D leads on Fashion-MNIST and CIFAR-10 S/L, including a 0.61-point advantage at L; LC leads on MNIST and CIFAR-10 M. Published values in Table II retain their original protocols, while reproduced baselines support the matched comparisons.

Table III reports single-seed convolutional test improvements of 3.26 and 2.08 points for S and M at unchanged architecture. Repeating full-S training over three seeds gives mean validation accuracies of 61.05% for LC and 59.82% for random connectivity. LC improves in two pairs, but the 1.23-point mean difference

TABLE I: Main evaluation configurations. b : thresholds per raw value; K denotes 1,000. Dense widths enumerate gates per layer; convolutional widths enumerate output channels with $k = 32/256$ for S/M.

Dataset/model	Layer widths	Gate functions	b	Updates	Batch
MNIST	$5 \times 1,334 + 1,330$	8K	1	108K	100
Fashion-MNIST	$5 \times 2,666 + 2,670$	16K	3	108K	100
CIFAR-10 S	$4 \times 12K$	48K	3	108K	100
CIFAR-10 M	$4 \times 128K$	512K	3	108K	100
CIFAR-10 L	$5 \times 256K$	1,280K	5	108K	100
Conv. S/M	$(k, 4k, 16k, 32k)$	83,552 / 668,416	3	350K / 200K	128

Dense: Adam, learning rate 0.01, no augmentation. Convolutional: AdamW, learning rate 0.02, decay 0.002, random crops with four-pixel padding and horizontal flips. All use raw gate logits and validation every 2K updates. Convolutional S/M share gate functions across 874,496/6,995,968 spatial gate applications; each classifier uses three dense layers of widths (1280k, 640k, 320k).

TABLE II: Dense test accuracy (T, %): mean $\pm$ sample SD over three paired seeds. Δ LC denotes the percentage-point gain over random connectivity.

Model	Gates, K	Random (REP)	LC (OUR)	LC-D (OUR)	Δ LC	Paired 95% CI	Random (R)
MNIST	8	91.27 $\pm$ 0.22	91.94 $\pm$ 0.14	91.91 $\pm$ 0.31	+0.66	[0.02, 1.31]	–
Fashion-MNIST	16	85.20 $\pm$ 0.26	85.72 $\pm$ 0.45	85.91 $\pm$ 0.36	+0.52	[−1.02, 2.06]	–
CIFAR-10 S	48	48.91 $\pm$ 0.35	52.10 $\pm$ 0.63	52.39 $\pm$ 0.30	+3.18	[0.77, 5.60]	51.27 $\pm$ 0.26
CIFAR-10 M	512	54.10 $\pm$ 0.17	58.65 $\pm$ 0.17	58.36 $\pm$ 0.25	+4.56	[3.78, 5.33]	57.39 $\pm$ 0.13
CIFAR-10 L	1,280	55.87 $\pm$ 0.31	60.46 $\pm$ 0.35	61.07 $\pm$ 0.46	+4.59	[3.72, 5.47]	60.78 $\pm$ 0.12

LC improves in all three seed pairs for every model. LC-D: dense specialization. R: published baseline [1]; exact-budget MNIST/Fashion references are unavailable.

TABLE III: Convolutional CIFAR-10 accuracy and training cost, with single-seed test results and three-seed validation replications reported separately.

Size	Method	Scope / seeds	Accuracy, %	Time, h	GPU, GiB	Random T (R)
S	Random (REP)	T / 1	57.37	4.98	1.831	60.38
S	LC (OUR)	T / 1	60.63	4.95	1.831	–
M	Random (REP)	T / 1	69.57	25.47	14.614	71.01
M	LC (OUR)	T / 1	71.65	35.69	14.614	–
S	Random (REP)	V / 3	59.82 $\pm$ 1.13	4.97	1.831	–
S	LC (OUR)	V / 3	61.05 $\pm$ 1.04	4.97	1.831	–

R: published S/M results [5]. Full-S V: paired gain +1.23 pp, 95% CI [−3.36, 5.82], 2/3 wins; extra seeds have no test results. Time: mean; memory: maximum peak allocation.

has a confidence interval of [−3.36, 5.82]. The test results therefore demonstrate a possible convolutional benefit whose magnitude and consistency remain uncertain. The additional seeds have validation results only.

C. Training memory and time

We compare LogicConnect with learned Top-32 routing using the M and L architectures and seven-threshold encoding of LILogic Net [4]. As shown in Table IV, LC achieves 52.54% and 60.19% test accuracy, improving on fixed random connectivity by 3.53 and 4.86 points. Top-32 achieves higher accuracies of 57.84% and 62.03%, respectively, but requires considerably more training memory. Its peak allocations are 8.10 and 24.86 GiB, compared with 0.474 and 1.557 GiB for LC. Consequently, LC reduces peak GPU memory by 94.2% and 93.7% and trainable parameters by 80%, at accuracy costs of 5.30 and 1.84 points.

Figure 2 illustrates the trade-off: at L, LC recovers 4.86 of the 6.70 points gained by Top-32 over random, approximately 73%, using one-sixteenth of its peak GPU memory. Recorded Top-32 training times are 2.59 and 18.98 times those of LC for M and L. These ratios include evaluation and reflect execution conditions, with only one Top-32 training seed. They quantify the observed cost difference but do not isolate an algorithmic speedup.

The fixed graph introduces a separate CPU construction cost before training. For the main dense M and L models, construction takes 14.33 and 31.22 seconds, compared with training durations of 41.39 and 113.53 minutes. This cost is below 0.6% of the respective training times, and peak GPU allocation remains equal to the random baseline. Full-S convolutional training averages approximately 4.97 hours for both methods at 1.83 GiB; full-M takes 35.69 hours for LC and 25.47 hours for random. Because

TABLE IV: Test accuracy and training cost under LILogic architectures: M uses $1 \times 64\text{K}$ gates and L uses $2 \times 128\text{K}$ gates, with seven input thresholds.

Size	Method	n	Accuracy, %	Params., M	Time, min	GPU, GiB	Accuracy (R)
M	Random (REP)	3	49.01 ± 0.43	1.024	14.62	0.474	49.17 ± 0.14
M	LC (OUR)	3	52.54 ± 0.30	1.024	14.59	0.474	–
M	Top-32 (REP)	1	57.84	5.120	37.81	8.100	57.28 ± 0.30
L	Random (REP)	3	55.33 ± 0.47	4.096	15.20	1.557	54.76 ± 0.27
L	LC (OUR)	3	60.19 ± 0.29	4.096	18.47	1.557	–
L	Top-32 (REP)	1	62.03	20.480	350.63	24.861	60.98 ± 0.19

Training: 35K updates, batch 256, Adam at 0.075, reflected-padding crops and horizontal flips. R: LILogic v2 [4]. Top-32 adds 4.096M/16.384M routing parameters.

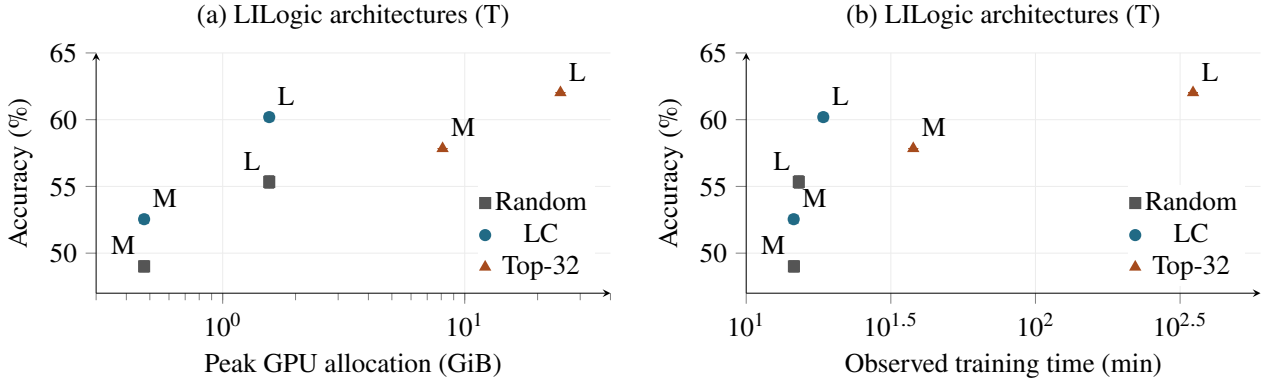


Fig. 2: Accuracy versus (a) peak GPU allocation and (b) observed training time under the LILogic M/L configurations. LC and random: three seeds, mean $\pm$ sample SD. Top-32: one seed. M/L identify architectures; wall-time ratios are descriptive execution measurements.

execution conditions differ, the M timing difference cannot be attributed solely to connectivity. Physical area and energy require a separate synthesis and implementation study.

D. Contribution of connectivity structure

To determine where the accuracy gain originates, we independently retain or randomize the first-layer and deeper-layer connections of dense M after defining the structured graph. Each randomization preserves every predecessor’s degree in each gate input position, separating pair identity from frequency of use. At 20K training updates, the SS, SR, RS, and RR configurations achieve validation accuracies of 59.13%, 58.61%, 54.64%, and 54.93%, respectively. The two letters denote first-layer and deeper-layer connectivity, with S indicating LC structure and R its degree-preserving randomization.

Figure 3 shows that first-layer structure contributes 4.49 points when deeper layers remain structured and 3.69 points when they are randomized. Both confidence intervals exclude zero. In comparison, adding deeper structure to the structured first layer yields 0.52 points with an interval of $[-0.52, 1.56]$, and the interaction interval also includes zero. The complete construction achieves the highest mean, but the clearest measured contribution comes from the first-layer pairing.

We further compare ancestry-based stage selection with a nominal scale schedule at 20K updates. LC changes validation

accuracy by -0.09 points for dense M and $+0.59$ for convolutional S, with intervals of $[-1.76, 1.59]$ and $[-1.12, 2.31]$. These comparisons do not establish an additional accuracy benefit from ancestry-based selection. A separate convolutional ablation finds a 2.14-point benefit from the structured body with a fixed structured classifier, with an interval of $[0.67, 3.61]$. The classifier was constructed using reference-body ancestry, so this finding applies to that particular body–classifier combination. All ablation intervals are exploratory and unadjusted for multiple comparisons.

E. Gate parameterization and transfer

We examine whether the connectivity benefit persists when gate functions are trained using an adapted WARP parameterization. Relative to random connectivity under the same parameterization, LC improves dense validation accuracy by 4.70 points with an interval of $[3.92, 5.48]$. The convolutional difference is 0.09 points with an interval of $[-1.24, 1.42]$. Both adapted parameterizations achieve lower accuracy than their corresponding gate-logit baselines, indicating that the connectivity and parameterization gains cannot be assumed to combine.

Finally, we evaluate the frozen CIFAR-10 models on CIFAR-10.1 [15] without adaptation. Dense M retains a 3.73-point LC

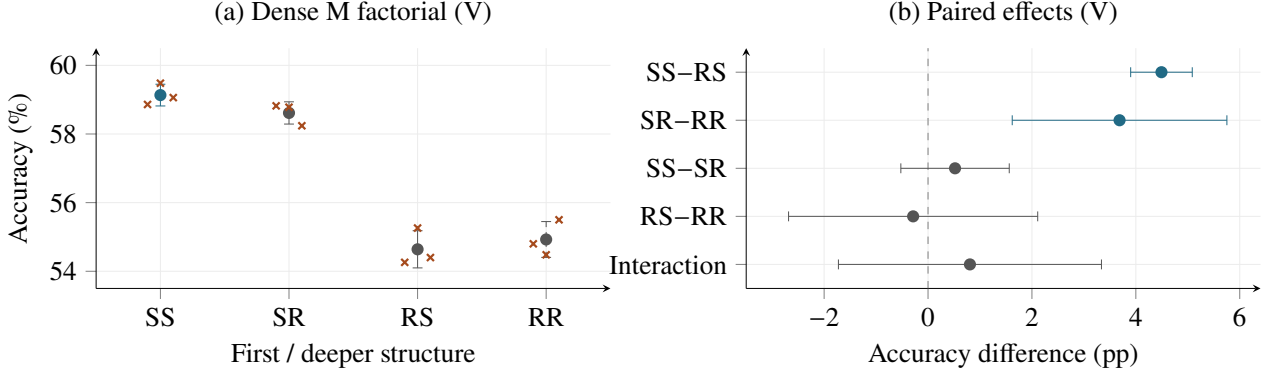


Fig. 3: Dense M attribution at 20K updates. S retains LC structure; R randomizes pairs while preserving each predecessor degree in each input slot. (a) Three seed values (crosses) and mean $\pm$ sample SD. (b) All five paired 95% intervals, exploratory and unadjusted. The first two contrasts isolate first-layer structure.

advantage across three seeds, with an interval of [2.80, 4.67]. Convolutional S and M change by -0.05 and $+1.95$ points, respectively, using one seed each; separate CIFAR-100 pilots do not establish an LC benefit. These findings extend the dense result to a related test distribution while showing that improvements across architectures and datasets remain dependent on the evaluated setting.

V. CONCLUSION

In this paper, we presented LogicConnect for designing fixed DLGN connectivity through semantic input pairing and multiscale matching. The method supports dense and convolutional architectures while preserving their logical budgets. Across three dense CIFAR-10 sizes, it improves test accuracy by 3.18–4.59 percentage points over random connections. Compared with learned Top-32 routing, it reduces peak GPU memory by 93.7–94.2%, at an accuracy cost of 1.84–5.30 points. Ablations identify first-layer pairing as the strongest measured dense contribution, while the benefit of deeper structure and ancestry selection remains unresolved. Further convolutional replications and physical implementation measurements are needed to establish the consistency of the convolutional gains and their circuit-level implications.

REFERENCES

- [1] F. Petersen, C. Borgelt, H. Kuehne, and O. Deussen, “Deep differentiable logic gate networks,” in *Advances in Neural Information Processing Systems*, vol. 35, 2022.
- [2] S. Bühner, A. Plesner, T. Aczel, and R. Wattenhofer, “BitLogic: A framework for gradient-based LUT-native neural networks,” *Transactions on Machine Learning Research*, 2026.
- [3] W. Mommen, L. Keuninckx, M. Hartmann, and P. Wambacq, “A method for optimizing connections in differentiable logic gate networks,” arXiv:2507.06173, 2025.
- [4] K. Fojcik, R. Zioma, and J. Armaitis, “LILogic Net: Compact logic gate networks with learnable connectivity for efficient hardware deployment,” arXiv:2511.12340v2, 2026.
- [5] F. Petersen, H. Kuehne, C. Borgelt, J. Welzel, and S. Ermon, “Convolutional differentiable logic gate networks,” in *Advances in Neural Information Processing Systems*, vol. 37, 2024.
- [6] S. Brändle, T. Aczel, A. Plesner, and R. Wattenhofer, “From MNIST to ImageNet: Understanding the scalability boundaries of differentiable logic gate networks,” arXiv:2509.25933, 2025.
- [7] S. Yousefi, A. Plesner, T. Aczel, and R. Wattenhofer, “Mind the gap: Removing the discretization gap in differentiable logic gate networks,” in *Advances in Neural Information Processing Systems*, 2025.
- [8] L. Rüttgers, T. Aczel, A. Plesner, and R. Wattenhofer, “Light differentiable logic gate networks,” arXiv:2510.03250, 2025.
- [9] L. Gerlach, T. Gerlach, L. Våge, E. Kauffman, and I. Ojalvo, “WARP logic neural networks,” arXiv:2602.03527, 2026.
- [10] Y. Kim, “Fitting multilinear polynomials for logic gate networks,” arXiv:2605.08657, 2026.
- [11] S. Lee, J. Sun, and J. Chung, “Two-stage unit tying for simplifying differentiable logic gate networks,” in *Proceedings of the International Conference on Machine Learning*, ser. Proceedings of Machine Learning Research, vol. 306, 2026.
- [12] Y. LeCun, L. Bottou, Y. Bengio, and P. Haffner, “Gradient-based learning applied to document recognition,” *Proceedings of the IEEE*, vol. 86, no. 11, pp. 2278–2324, 1998.
- [13] H. Xiao, K. Rasul, and R. Vollgraf, “Fashion-MNIST: A novel image dataset for benchmarking machine learning algorithms,” arXiv:1708.07747, 2017.
- [14] A. Krizhevsky, “Learning multiple layers of features from tiny images,” University of Toronto, Tech. Rep., 2009.
- [15] B. Recht, R. Roelofs, L. Schmidt, and V. Shankar, “Do CIFAR-10 classifiers generalize to CIFAR-10?” arXiv:1806.00451, 2018.